

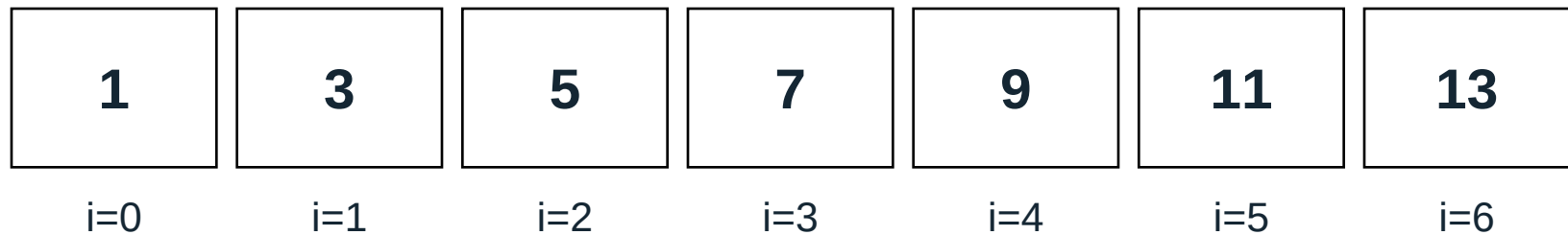
1. Sorted input and candidate interval

Precondition: a is a finite nondecreasing array.

For valid indices $i < j$, $a[i] \leq a[j]$.

Goal: return any index of x , or -1 if x is absent.

Candidate interval: $[l, r]$, with both endpoints included.



Example: $x = 11$; initially $l = 0$, $r = 6$.

The boxes show values; i labels zero-based indices.

2. Why discarding half is safe

Invariant: if x occurs, an occurrence remains in $[l, r]$.

Initially: $[0, n - 1]$ contains every valid index.

Choose $m = \text{floor}((l + r) / 2)$.

If $a[m] < x$: for $i \leq m$, $a[i] \leq a[m] < x$.

Therefore set $l = m + 1$. No occurrence is discarded.

If $a[m] > x$: symmetrically set $r = m - 1$.

If $a[m] = x$: return m .

Each unequal step strictly shrinks the interval.

When $l > r$, no candidate remains; return -1 .

3. Algorithm and worked trace

```
l, r = 0, len(a) - 1
while l <= r:
    m = (l + r) // 2
    if a[m] == x:
        return m
    if a[m] < x:
        l = m + 1
    else:
        r = m - 1
return -1
```

x = 11: [0, 6] -> m = 3, a[m] = 7 -> l = 4

[4, 6] -> m = 5, a[m] = 11 -> return 5

Empty input returns -1. Duplicates: any match is valid.